> This is an **optional final project**.
>
> 1. Students can opt to complete this project **instead** of the final exam.
> 2. Students **can't** do both end-of-course-assessments and take the best grade. It's one or the other.
> 3. Students can partner for this project **in the same group** they completed ZOBOS in, otherwise students must complete this project independently.

In this assignment you will produce the final piece of a simple compiler using a course specific instruction set. The name of your application must be CZAR (case sensitive). CZAR will be able to generate code for particular machines and software architectures, its input will be a decorated AST from the ZOBOS project, and it will write its output to a file in a form similar to common assembler files.

> Like ZOBOS, this project's requirements morph from semester to semester, so some parts of this write-up may not be pertinent to **this semester's** project requirements. *Read accordingly!*

CZAR Features

Your CZAR project will generate assembler for these source features:

Feature ID	Description
Extra credit features are a shade of green and marked with a**	
Undergraduate submissions	
GLOBAL	Implement global variables
INTEXPR	Implement integer expressions, including `emit` and `rand` for integer and `bool` arguments
FLTEXPR	Implement floating point expressions, including `emit` and `rand` for floating point arguments
BRANCH	Implement `if-then` and `if-then-else` branching structures (requires the Jumps step).
LGVS**	Implement large global values (too large for immediate instructions) and `string` values; implement `emit` for string.
~~GFUNCS**~~	~~Implement functions that take no parameters and have no local variables (aside from the implicit return value of zlang at "parameter" scope).~~

Graduate submissions	
LOOPS	Implement `do-while` and `while-do` loops (requires the Jumps step).
~~SFUNCS~~	~~Implement functions that pass all their arguments and return values on the stack with multiple levels of local scopes.~~
LGVS**	Implement large global values (too large for immediate instructions) and `string` values; implement `emit` for string.
Unused this semester	
LFUNCS	Implement functions that take no parameters but do have a single scope level (namely 2) of local variables (aside from the implicit return value of zlang at "parameter scope" of 1).
AFUNCS	Implement functions that pass all their arguments and return values in registers with multiple levels of local scopes.
MIXEXPR	Implement mixed domain integer and floating point expressions, Boolean AND, OR, NAND, XOR and complement operations.

Machine and Software Architecture Description

Instructions, integers, floating point values and Boolean values are 4 bytes each; these types should be stored in memory with 4 byte alignment. zlang constant string data are 1 byte for each character and can be stored at any memory location (1 byte alignment).

Registers

$R_N \geq F_N \geq 4$ are the absolute number of general purpose and floating point registers in the CPU. Registers use index notation so the first general purpose register is `R0`.

- General purpose and floating point values (registers) are 4 bytes each and must be 4 byte aligned in memory.
- You may decide which registers to use as **work registers**.
- All other registers are the **allocatable registers** for the treeCG algorithm.

Immediate Ranges

Immediate integer values range is $[-16384, 16383]$ floating point immediate values are in $[0, 1310.71]$ and support only two decimal places worth of precision.

Function Calls

This semester parameters and return values are passed via the stack.

Caller ($\mathcal{R}$) saved registers (pushed onto the stack in the *prologue* of a `CALL` AST node,

popped in the epilogue):

- Any parameters to the caller passed via registers (possibly not a code requirement this semester).
- The caller's FP
- The caller's RA

Call**ee** ($\mathcal{E}$) saved registers in (pushed onto the stack in the *prologue* of the `FUNCTION` AST node, popped in the epilogue):

- General purpose and floating point registers the callee (might) use in expression value calculations (**except for work registers** which by definition the caller should have no expectations of value preservation.).
- The callee's RA.

Input

Argument	Value
1	R_N, F_N
2	`program.tok`
3	`program.def`
4	`program.czr`

Register Counts (parameter 1)

Example values for CZAR's first command line parameter might be:

- `16,16`
- `16,8` or
- `8,5`

Token Stream (parameter 2)

`program.tok` is a **token stream** generated by LUTHOR and used as input to ZOBOS. If you would like to continue on with your ZOBOS code base for implementing machine code generation, you can simply parse this token stream file as in ZOBOS. It turns out pretty much everything *needed* to be done *can be done* with additional **visitor passes** and more `(key,value)` pairs left behind in the augmented AST.

Once you have your AST constructed with your pre-existing ZOBOS functionality, perform a (perhaps slightly modified) symbol table pass, and then you're off to the races with generating machine code.

> There are no **syntax** or **semantic errors** in the program token stream. Finding these was the job of ZOBOS, now we focus on generating

machine instructions.

If you need the zlang grammar file or `zlang.lr` SLR(1) table data for parsing, do as in ZOBOS submissions and provide them alongside your `Makefile` or `Build.sh`.

Annotated AST (parameter 3)

If you'd prefer **not** to integrate CZAR into your ZOBOS code base,[1] your CZAR can read the `program.def` third parameter file and construct an AST of the program source without having to compile or use SDT.

Details for the `.def` file format are here.

Once you have the AST built, you'll need to run a symbol table constructing visitor pass and then you'll be ready for code generation.

CZAR Output (parameter 3)

The goal of this project is to generate a memory layout in the form of a common assembler input file. The instruction set language we'll use is specific to this course but no different than many other instruction sets and input file formats (at least in its fundamentals). This is the referred to as the **blaster** language, in deference to our glorious school mascot.

The output file `program.czr` *contents* are **case insensitive and line oriented**, empty lines allowed. The output file's name is, of course case, sensitive.

An annotated grammar file for the blaster language is here, for the overly curious a raw CFG file is also available.

I think it is most efficient to learn the instruction set `.src` file by `.src` file. Which is to say as you are developing and testing your CZAR, use the example resources (the `.src` and `.czr` files) to see the details of the instruction set language. It is **very** similar other RISC instruction sets and you'll probably be able to get pretty far just on your previous knowledge.

When you run into an instruction whose arguments or syntax don't make sense, **then** go to the commentary in the annotated grammar file for clarification. The places where the blaster language deviates from standard RISC patterns make the assembler instructions easier to generate from typical AST representations (IMHO).

Eventually, though, you'll probably want to see the details for:

- **Immediate values** (`ImmVal`, `NegImmVal`, `ImmFloat`), right at the top of the file
- And the several possible notations for **Addresses** (about line 50)

Basic I/O Tests

As is usually the case, CZAR should have a non-zero exit status if the input file your CZAR uses (`program.tok` or `program.def`) or the output `program.czr` does not exist or cannot be written.

> You only need to check the input file your CZAR uses, you can safely ignore the other without diminution of your project grade.

Coding up CZAR

Each step is given a "**StepId**" to aid in write-up clarity, but also (hopefully) to make it easier to ask specific questions about the code generation process.

1. **GenAST** (Of course) parse `program.tok` or read in `program.def`; then fire-up your symbol table code!
2. **Symbols** Traverse the tree with a visitor pass, **you do not need to look for any semantic errors**! You probably have this logic already implemented in your ZOBOS project.
    a. Collect global symbols into your symbol table
    b. If you're implementing large global values this semester, see also this page.
    c. If you are generating code for functions this semester, also do these steps.
3. **dataSeg** Now we know all the global variables in the program. The next step is to layout the **DATA segment** of the program's memory map. The data segment will begin at base memory (address 0) and grow upwards.

    > A diagram of CZAR's memory map organization is at the **imageData** step. It **might** be different than presented in show_stack-management.pdf (depending on the semester requirements), don't let it concern you.

    a. For each global identifier in the symbol table, calculate its location and size. Augment each identifier's symbol table entry with its absolute memory location.

        > Remember that data types should be **aligned to their size**. So a four byte `float` should be at a memory address divisible by four.
        >
        > `string`s can be stored at any offset.

    b. If you're implementing code for **functions** this semester, do these steps as well.
    c. Remember the total size (in bytes) of the data segment calculated, or have some easy way to determine this value.
4. **registerNeeds** Next, run a `registerNeeds` visitor pass on the AST. This should label each expression tree node with a `.regCount` value as in registerNeeds.pdf.

a. Manage the `assign` (=) binary operator (see your group's lga-code-generation solution for a start)
b. `registerNeeds` can be adjusted to account for **immediate arithmetic instructions**, which the `blaster` language has plenty of (`LD R7 #1`, `+ R3 R3 #10`, `MUL F0 F1 #1.23`, ...). See your group's lga-register-allocation solution for a start.

> zlang has no mechanism to read data from inputs (keyboard, disk, network) so you have to implement at least *immediate instructions* in order for computations to be performed (well, at least computations on non-garbage values).
>
> A real compiler would also implement large literal values, which might be an extra credit option this semester.

c. If this semester's CZAR requires support for mixed domain expressions, follow these instructions to track the "register domain" to use at each node.

5. **Frames** (If function implementation is required for the semester's CZAR, see this page .)
6. **treeCG** Now use the `treeCG` algorithm (or one of its variants if you're using virtual registers) as a basis for generating the instructions required for each expression tree. You can accumulate these instructions in a list or vector and "carry" them up to the expression tree root (storing them there), or you can hang smaller chunks of instructions at the expression tree nodes (just be sure a subsequent traversal of the AST "sees" these in the correct instruction order).

> The latter is advisable if you're implementing `LOOPS` or `BRANCH` features in CZAR. More details in Jumps below.
>
> Of course you will need to augment `treeCG` with some critical logic for our source language and our target instruction set

a. When `LOAD`ing global values (scope 0), use their memory locations calculated in **dataSeg** and stored with the identifier in the symbol table
b. Handle `assign` (`=`) nodes with `STORE` instructions.
c. Handle **immediate arithmetic operations** such as `+ R3 R3 #x0f` and immediate `LOAD`s such as `LD R4 #2`
d. Handle the `emit floatexpr` and `emit intexpr` zlang statements. Use the correct type of register (R? or F?) to select proper interpretation of the register bits and formatting of the output value.

> **There are no** instructions issued for `emit symtable` or `domain()` statements. These language constructs were just a hook for ZOBOS grading.

e. If you are managing strings generate instructions for `emit id, offset, length` program statements. In zlang the `id` should be a `string` identifier, but in our assembly the `emit` instruction looks for a string

address as the first argument and the *offset* and *length* in two general purpose registers. Some examples:

```
emit @4w, R3, R4
```

or

```
load R2, #4w
emit @R2, R3, R4
```

f. If this semester's CZAR requires support for mixed domain expressions, handle `CAST` operations in an expression tree with a `CONV` instruction. See here for more details.

g. If this semester's CZAR requires function support, see this page.

7. **Jumps**

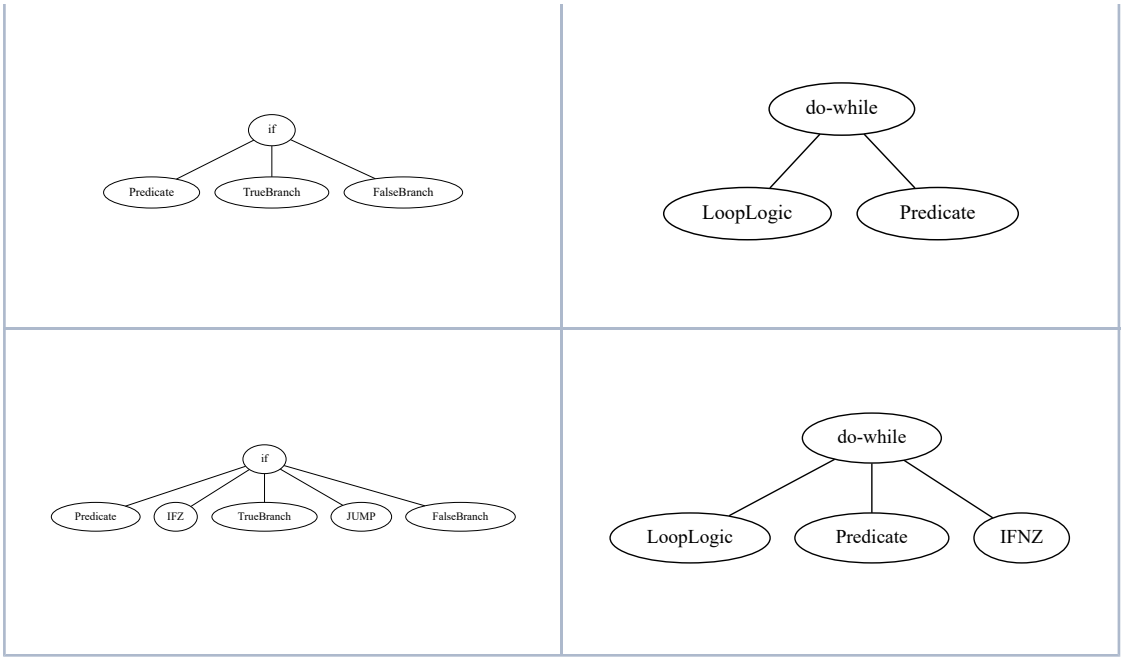
> Check this semester's CZAR requirements, you may not be required to handle all or any control structures this semester; or they might be an extra credit opportunity.

The next step is to traverse the AST searching for **control structures** such as if-then branches or loops. When a control structure is encountered, add the appropriate `JUMP`, `IFZ` or `IFNZ`[3] nodes to the AST control structure, and tag this node with the correct machine instruction using a **relative PC jump** argument. See your learning group's lga-code-generation solution for an example of a `while-do` pre-test loop and the PC codes lecture slides for an example of branching logic.

> How do we know how many instructions to jump? Because the previous **treeCG** step generated the instructions for all the expressions in the source,
>
> i. either traverse the AST and count them, or
> ii. modify your treeCG code to carry the number of instructions up from the leaves to the root of the expression.

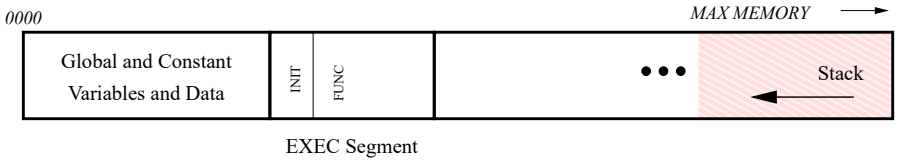
> For the `IFZ` and `IFNZ` instructions, how do we know which register should be evaluated for the conditional branch?
>
> Thanks to **register targeting**, we know it will be the first register in the `.regList` provided to the `PREDICATE` node of the control structure during the **treeCG** step.

if-then-else	do-while



> Immediate range limitations apply to PC relative jumps like `JUMP #8i(PC)` or `IFZ R7,#14i(PC)`. But **none of the testing source** for the CZAR project should trip over these limitations.

8. **imageData** This is it! You can begin to write the memory image of the compiled program. We start with the *constant data* and *global variables* portion of the memory image:



DATA

where global variables, implicitly `const` `string`s (global and within function scopes), `const` local variables in functions, and implicitly constant synthetic variables (u_is from the **Symbols** step)

EXEC

all program instructions

INIT

instructions at global scope such as

```
const int i = 3;
string msg = "hello world";
if ( i < 6.67e-11 ) emit msg, 0, 11;
string msg = "that's all folks!";
```

FUNC

function instructions (if required this semester)

To generate the **DATA** segment instructions, look into your symbol table that was augmented in the **dataSeg** step, if there are any *constant global literals* in scope 0 (and this is the only scope they would be in...), then issue a `data` instruction for them:

Symbol	Type	Value	Address	`data` instructions
`i`	`int`		0	`label @0 i`

`!0`	`stringval`	"hello world"	4	`label @4 !0`
				`data @4`
				`:hellox20world:`

`!1`	`floatval`	6.67×10^{-11}	16	`label @16 !1` `data @16 #6.67e-11`

Some explanations are required here:

a. The `label` instruction allows you to name a memory address. This can make error and status messages from the machine emulator used in the course (`emule`) easier to interpret.
   i. `label` instructions are not required.
   ii. `label` instructions **are not machine executed operations** they are meta data about the memory map. So `label` doesn't consume space in memory.

b. The global variable `i` doesn't have a `data` statement because it isn't a `string` and it is being initialized to an arithmetic expression. It's value will be determined in the **imageInit** step (and this, only if it doesn't hold a garbage value).

c. Since `label`s are optional, **you might not write any instructions at this stage!** In fact the **remaining instructions written in this example are only needed** if you are implementing *constant global literals* in the **Symbols** step.

d. The `data` instruction takes an absolute address preceded with `@` and then a data value. Numbers are provided in immediate format (beginning with `#`) and string values in alphabet encoding delimited by colons (`:`) on either side.

e. Keep in mind that we are writing the **memory image** with `data` statements! The `#6.67e-11` is in the syntax of immediate notation, but in this case the actual floating point pattern for the value is going to be stored at address 16 --- it's not being embedded into an instruction and thus not limited by the instruction set's immediate range limitations.

9. **imageInit** Now we generate the instructions for globally scoped code.
   i. First, we tell the machine where these instructions should be written in memory with the `init` statement. Determine the size of your **dataSeg**, pad it for alignment to the machine's instruction size, and write:

```
init @24
```

In this example, the value of 24 is because the last constant value was

6.67×10^{-11} at address 16, and our machine is using 8 byte floats.

> If there are no global variables, and you aren't managing *constant global literals*, then you would write `init @0`. Because there is no data segment to the memory map.

ii. Now traverse the AST **ignoring FUNCTION nodes** and write out the instructions generated for all expression evaluations, control structures, `emit` and `rand` statements. Essentially, all **treeCG** generated instructions at global scope.

iii. Finally, write a single `return` statement (the initialization code hands control of the CPU back to the machine (OS?) when it is done.

10. **imageFunctions** (If function implementation is required for the semester's CZAR, see this page .)

I am providing to students the same grader script that will be used on gradescope for testing your `CZAR`. Here is how to use it:

First, fire up your usual course development environment and download this tarball to your development environment and unroll it in a temporary directory.

```
$ ls czar-student.*
czar-student.tar.bz2
$ mkdir ~/tmp
$ cd ~/tmp
$ tar xjf ../czar-student.tar.bz2
:
$ ls
czar/
```

Now go to the directory holding your `CZAR` and execute the `grader.sh` script from the `czar-student.tar.bz2` resource.

```
$ cd ~/compilers/czarCC
$ ls CZAR
CZAR
$ ~/tmp/czar/grader.sh
:
:
:
```

If you are enrolled for graduate credit, set the `GRADERMODE` environmental variable to `ALL` before running the grader script. The grader script will then test against all CZAR features for this semester specified in the features table. The default (if `GRADERMODE` is not found in the environment) is `GRADERMODE=UGRAD` is to test against only the undergraduate targets.

```
$ GRADERMODE=ALL ~/tmp/zobos/grader.sh
```

You will need to read any messages from the script carefully, and you may need to hit `ENTER` several times throughout its course. This script checks for:

a. missing `.tok` or `.def` input file (you need to check **only** the input file you intend to

use).

b. truncated (invalid) `.tok` or `.def` input files (again, check **only** the input you will use).
c. required tests for this semester's CZAR
d. extra credit tests for this semester's CZAR (if there are any)

The latter tests results are displayed on the terminal screen, but they whip by pretty quickly so you may need to scroll up and make sure you see them all.

When there is a discrepancy in expected results the script either:

- points you to data files or directory along side your `CZAR` where more failure details are available,
- or all the details are displayed on screen.

Before the `grader.sh` terminates, it will show a summary of some specific rubric line results for the programming project.

> The grader will use this very same script, along with some additional testing data to check your submitted work. If your `CZAR` flies through without a hitch, you can likely suspect a good grade for the assignment.

What is grader.sh doing?

`grader.sh` runs your `CZAR` program with the appropriate inputs like any other course project. However, depending on the *work registers* you choose, the order of your *allocatable registers* in **treeCG**, and a myriad of implementation choices, it is unreasonable to expect a perfect one-to-one correspondence of assembler instructions between any two CZAR implementations.

Instead, a machine emulator for the blaster assembler language is available in `${COMPGRADING}/emule`[4] [5] and your `CZAR` assembler output is "run" by this emulator. `emit` statements are automagically turned into OUTPUT compliant data on `stdout`. So, your `CZAR` assembler is essentially graded on producing the expected output from any of the original `*.src` files. Congratulations, BTW, you've written a compiler 🙂

Submit Your Work

Partners

You may complete this assignment in a group of two students **that are both rostered for the same course (undergraduate or graduate credit)**.

Checking and Submitting Your Work

Here are some things to double check your submission against:

i. Your archive does not raise errors when "unrolling" with the `${COMPGRADING}/explode-subm` script.

```
$ echo $COMPGRADING
/COMPGRADING
$ ls
my-project.zip
$ "${COMPGRADING}/explode-subm" -P PROJECT my-project.zip ./here
```

ii. `PROJECT` is different for each project, the first two (for example) are `ALPHABETENCODING` and `LUTHOR`.

iii. Your archive contains only the essential files, **don't provide unneeded files**

iv. Don't provide files that have been provided to you (such as test cases in the `*-student.tar.bz2` files); that would just be silly.

v. Make sure the `grader.sh` script for the assignment runs to completion.

> You aren't required to use `.zip` files, `explode-subm` knows about the usual suspects: `.tar`, `.tar.gz`, `.tar.bz2` ...

When you are happy with your work, submit your project archive file for grading at https://www.gradescope.com.

Rubric

This work is worth 60 points for students enrolled in the undergrad course and 65 points for students enrolled in the graduate course. There are 5 points of extra credit, the adjustment will be made during course grade calculations.[6] `CZAR` satisfies the "final exam" component of the Syllabus..

Requirements	Points	Notes
Meets compilers course project requirements	10	
Basic I/O tests	5	
Integer arithmetic instructions, `emit` and 'rand' for integer values (`intexpr-?.src`[7])	20	
Floating point arithmetic instructions, 'emit' and 'rand' for float point data types (`fltexpr-?.src`)	20	
if-then, if-then-else control structures implemented (`ifexpr-?.src`)	5	
(GRAD) do-while, while control structures implemented (`loopexpr-?.src`)	5	
EXTRA CREDIT Constant global literal data implemented (`cgldata-?.src`)	5	

1. **There are good reasons not to do this, pretty much disjoint from the good reasons to continue on with your current ZOBOS codebase. (1)**
2. **All the commentary is of course, useful by definition `:)` (2)**

3. **These node names refer to the instruction set's PC manipulation instructions. (3)**
4. **Get it? Blaster the Mines mascot has an eMULEator! Woot! (4)**
5. **Type `${COMPGRADING}/emule --help` if you're curious (5)**
6. **Do the math, and you'll see the 5pts translate to *about* the point cost of skipping `ZOBOS` for those of you that needed to. (6)**
7. **Examples for each rubric line item follow naming conventions. (7)**

Assignments/CZAR (last edited 2026-05-07 21:17:42 by khellman)